

Kế hoạch triển khai — Auto Pipeline Video AI

Kế hoạch triển khai — Auto Pipeline sản xuất Video AI

Tài liệu này phân tích Quy_Trinh_Auto_Pipeline_Full11.docx (bao gồm 4 comment góp ý trong file) và đề xuất giải pháp + định hướng triển khai chi tiết.

1. Tóm tắt tài liệu gốc

Kết luận đã chốt trong doc: Hệ thống có thể auto 100% về mặt kỹ thuật (từ đọc kịch bản → xuất video), nhưng về nghệ thuật/chất lượng nên dùng mô hình **Hybrid Agentic Workflow** — tự động hóa qua MCP, có User duyệt (Human-in-the-Loop) tại 3 điểm chốt.

Quyết định kiến trúc đã chốt (2026-09-19): điều khiển hợp nhất qua 1 tool duy nhất. Toàn bộ pipeline (5 bước) được điều khiển từ **một Dashboard web duy nhất**, sắp xếp theo đúng thứ tự bước, mỗi khâu có bộ nút điều khiển riêng; người dùng không phải chạy qua lại giữa nhiều công cụ (xem Mục 3.4). Triển khai 2 giai đoạn: V0 (thử nghiệm qua Claude Desktop + MCP) rồi V1 (Dashboard hợp nhất) — xem Mục 7.

Quy trình 5 bước:

Bước	Thành phần kỹ thuật	Điểm kiểm duyệt	Mức tự động
1. Input kịch bản & phân tích	Claude Director + MCP Script Parser + SQLite	ĐIỂM 1: duyệt Character Bible & bảng phân cảnh	Bán tự động
2. Sinh ảnh cảnh hàng loạt	MCP Deepix Connector (REST API)	ĐIỂM 2 (quan trọng nhất): duyệt ảnh Approve/Reject	Tự động 100% (can thiệp khi cần)
3. QC ảnh & sinh video motion prompt	Claude Vision + Director Prompt	ĐIỂM 3: duyệt nhanh prompt video	Tự động 90%
4. Sinh video từ ảnh	MCP Clip AI / Kling AI API	Không bắt buộc dừng	Tự động 100%
5. Ghép & render video	MCP FFmpeg Studio (local)	Điểm chốt: nghiệm thu cuối	Tự động 100%

Lộ trình đề xuất trong doc (4 tuần): (1) hạ tầng & API, (2) build 4 MCP servers (project-db, deepix, clipai, ffmpeg), (3) Streamlit dashboard + dry-run, (4) đóng gói 1-click startup + bàn giao prompt templates.

2. Phân tích các comment (rất quan trọng — đây là các gap chưa được giải trong bản doc)

Comment 1 — Cần State Machine cho toàn bộ pipeline

"Nên bổ sung State machine... queued/running/succeeded/failed/retryable(?)/rejected/cancelled — User ở khâu bắt buộc review có thể bắt kịp trạng thái và đưa ra quyết định hợp lý hơn."

Gap: Doc mô tả trạng thái rời rạc kiểu tự do (`READY` , `IMAGE_GENERATED` , `VIDEO_PROMPT_READY`) nhưng không có một state machine thống nhất, không có trạng thái lỗi/hủy/retry rõ ràng cho từng job (ảnh, video, render).

Giải pháp: Định nghĩa 1 bảng `job_state` dùng chung cho mọi loại tác vụ (scene, image_gen, video_gen, render), với state enum cố định: `queued` → `running` → `succeeded` | `failed` → `retryable` → (`queued` lại) | `rejected` (user reject) | `cancelled` . Mỗi transition được ghi log (audit trail) trong SQLite để Dashboard hiển thị real-time, và để Claude/MCP biết chính xác việc nào cần resume, retry, hay dừng hẳn.

Comment 2 — Mâu thuẫn về mức độ auto ở bước QC ảnh

"Đây là bước quan trọng nhất nên mức độ tự động không 100%... Trong trường hợp ưu tiên tự động 100%, user review không quan trọng. Trong trường hợp quan trọng thì cần can thiệp bắt buộc vì job phát sinh sau reject hủy/tạo lại như thế nào."

Gap: Doc ghi Bước 2 là "Tự động 100%" nhưng đồng thời gọi đây là "ĐIỂM KIỂM DUYỆT... QUAN TRỌNG NHẤT" — mâu thuẫn nội tại. Chưa định nghĩa rõ: reject thì job con (regenerate ảnh) được tạo lại như thế nào, ai sở hữu retry count, khi nào dừng hẳn.

Giải pháp: Tách rõ 2 chế độ vận hành qua `operating_mode` (config theo project/khách hàng), dùng chung 1 bước QC scoring — khác nhau ở **ai có quyền quyết định chuyển state**:

- **auto** — QC Agent (model AI chuyên duyệt ảnh, tách biệt với Claude Director viết prompt) tự chấm điểm và **tự quyết định** pass/fail theo `auto_pass_threshold` . Score đạt → job tự chuyển `APPROVED_AUTO` → tự động sang Bước 3/4, không ai cần bấm gì.
- **human_qc** — QC Agent vẫn chấm điểm như trên, nhưng chỉ mang tính gợi ý/pre-filter (sort ảnh theo score để user duyệt nhanh hơn). Mọi ảnh dừng ở state `pending_human_review` ; chỉ khi **user bấm Approve** job mới được phép đi tiếp.

Khi Reject (dù do QC Agent tự động hay user): tạo `job` mới `image_gen` với `parent_job_id` trở về job cũ + ghi `retry_reason` , tăng `retry_count` . Nếu `retry_count` vượt `max_retry_count` (vd 3 lần) → tự động chuyển state `failed` và **escalate cho user** — đây là van an toàn bắt buộc ngay cả ở mode `auto` , tránh lặp vô hạn tốn credit.

Mọi quyết định (AI hay người) đều ghi vào `review_log` với cột `reviewer_type` (`ai_agent` | `user`), để sau này so sánh QC Agent quyết định đúng bao nhiêu % so với user thật — dữ liệu này dùng để tinh chỉnh `auto_pass_threshold` hoặc làm căn cứ chuyển hẳn project sang mode `auto` khi đã đủ tin cậy.

Comment 3 — Ngưỡng auto-pass của Claude Vision chưa rõ

“Có ngưỡng để Claude Vision cho auto-pass không? User có thể quy định/chỉnh sửa ngưỡng cho Claude hay quá trình ảnh hưởng này do lựa chọn mô hình?”

Gap: Chưa có cơ chế cấu hình ngưỡng QC (confidence score, checklist tiêu chí) — hiện tại là “hộp đen”.

Giải pháp: Xây `qc_checklist` cấu hình được (YAML/DB) gồm các tiêu chí cụ thể Claude Vision chấm điểm (vd: đúng nhân vật theo Character Bible, không lỗi tay/mặt, đúng bố cục cảnh, đúng mood màu sắc) — mỗi tiêu chí trả về score 0-1. `auto_pass_threshold` là 1 con số user chỉnh được trong Dashboard (per-project), ví dụ ≥ 0.85 tự pass, 0.6–0.85 vào hàng chờ review, < 0.6 tự động reject + note lỗi cụ thể. Việc chọn model (Claude Vision version) cũng ảnh hưởng độ chính xác score — nên version model dùng phải được pin (không auto-update) để ngưỡng không bị trôi.

Comment 4 — Batch lớn (>100 video) & vấn đề bản quyền IP bị chặn (Kling AI)

“Khi tạo 1 batch request lớn (>100 videos)... cần heartbeat check hoặc always allow... Mô hình Kling video 3.0 omni đang ban một số IP (vd Wonder Woman) → Failure to pass risk control system → User nên check trước nhân vật/IP trước khi request gen nhiều video bị fail.”

Gap kếp: 1. **Vận hành/polling:** batch lớn cần cơ chế polling dài hạn không cần user ngồi approve từng request. 2. **Rủi ro nội dung:** nhân vật có thể trùng IP bản quyền (superhero, celeb...) bị Kling AI content-moderation chặn → tổn credit, tổn thời gian mà không biết trước.

Giải pháp: 1. MCP Clip AI Connector chạy **heartbeat polling background** (interval cấu hình được, vd mỗi 60-120s) tự động check trạng thái job trên Kling/Clip AI, không cần user “allow” từng lần — chỉ cần approve 1 lần ở đầu batch (long-lived credential/session), Dashboard hiển thị tiến độ real-time (x/100 done). 2. Thêm bước **Pre-flight IP/Content Check** ngay ở Bước 1 (Character Bible): Claude quét mô tả nhân vật, cảnh báo nếu tên/mô tả trùng IP nổi tiếng đã biết bị risk-control chặn (duy trì 1 blacklist cập nhật thủ công/định kỳ từ thực tế fail logs). Nếu phát hiện, cảnh báo sớm cho user **trước khi** chạy batch gen ảnh+video hàng loạt, tránh gen hàng loạt rồi fail hàng loạt ở Bước 4. 3. Ghi log mọi lần fail vì risk-control vào bảng riêng (`content_moderation_failures`) để dần xây blacklist nội bộ từ dữ liệu thực tế.

3. Giải pháp & định hướng tổng thể (đã tích hợp comment)

3.1 Kiến trúc dữ liệu (SQLite schema đề xuất)

- `scenes` — 1 record/cảnh, cột `state` theo state machine chung (comment 1)
- `jobs` — mọi tác vụ async (image_gen, video_gen, render) đều là 1 job: `id`, `type`, `scene_id`, `parent_job_id`, `state`, `retry_count`, `retry_reason`, `created_at`, `updated_at`
- `qc_results` — điểm QC theo tiêu chí (comment 3): `job_id`, `criterion`, `score`, `threshold_at_time`, `auto_decision`

- `content_moderation_failures` — log fail vì IP/risk-control (comment 4)
- `review_log` — audit trail mọi quyết định (comment 2): `job_id`, `reviewer_type` (`ai_agent|user`), `decision`, `note`, `decided_at`

3.2 Cấu hình vận hành (theo project, chỉnh trong Dashboard)

- `operating_mode`: `auto` | `human_qc` (comment 2) — xem bảng so sánh dưới đây
- `qc_auto_pass_threshold`: float, mặc định 0.85 (comment 3)
- `qc_agent_provider`: **Claude Vision** (system prompt QC riêng, tách biệt system prompt của Claude Director) — quyết định 2026-09-18, xem lý do so sánh ở dưới
- `qc_model_version`: pin cố định (version cụ thể, không auto-update) để ngưỡng không bị trôi
- `batch_poll_interval_seconds`: mặc định 90s (comment 4)
- `max_retry_count`: mặc định 3

So sánh 2 giá trị `operating_mode`:

	auto	human_qc
QC Agent chấm điểm	Có	Có (chỉ để gợi ý/pre-filter)
Ai quyết định pass/fail	QC Agent (theo <code>auto_pass_threshold</code>)	User
Pipeline có block chờ Bước 3 không	Không (trừ khi escalate sau <code>max_retry_count</code>)	Có, luôn chờ user Approve
Khi nào cần user can thiệp	Chỉ khi retry vượt ngưỡng (safety net)	Mọi ảnh

3.2.1 Vì sao chọn Claude Vision làm `qc_agent_provider` (so với GPT-6 Astra)

Đã cân nhắc GPT-6 Astra (OpenAI, ra mắt 09/2026, \$10/\$50 mỗi 1M token input/output) nhưng chọn Claude Vision vì: 1. **Đồng bộ vendor** — Claude Director (Bước 1, 3) đã dùng Claude Enterprise; dùng chung Claude cho QC Agent giảm số vendor, đơn giản vận hành, đúng tinh thần “1-Click Startup” trong lộ trình gốc. 2. **Chi phí phù hợp bài toán** — QC checklist là tác vụ phân loại/so khớp (đúng nhân vật, lỗi tay, bố cục), không cần model reasoning nặng như GPT-6 Astra; chạy hàng trăm-ngàn ảnh/ngày ở mức giá reasoning cao sẽ đội chi phí không cần thiết. 3. **Data governance** — không phát sinh thêm đường dữ liệu ảnh ra vendor thứ 2, quan trọng vì pipeline xử lý nội dung có rủi ro bản quyền IP (Comment 4).

`qc_agent_provider` vẫn để dạng config swap được — nếu sau này `review_log.reviewer_type` cho thấy Claude Vision chấm sai lệch nhiều so với quyết định thật của user, có thể A/B test model khác mà không phải đổi kiến trúc.

3.3 Kiến trúc & MCP Servers (6 server + lớp core dùng chung)

Kiến trúc lớp (để chuyển V0 → V1 không phải làm lại): - **Core Python dùng chung** — toàn bộ logic nghiệp vụ (state machine, gọi Deepix/Clip AI/FFmpeg/nhạc, retry, ghi log) nằm ở đây. - **MCP servers** — chỉ là vỏ mỏng bọc core, phục vụ V0 (gọi từ Claude Desktop). - **Dashboard (V1)** — gọi cùng core; là **frontend orchestrator duy nhất**, không có đường thao tác nào khác (không mở SQLite tay, không gọi API tay,

không chạy script CLI riêng). - **LLM runner hoán đổi được** — bước cần Claude (Director Bước 1/3, QC Bước 2, Music Brief) đi qua 1 lớp runner: V0 = chạy thủ công qua chat Claude Desktop, V1 = Anthropic API; dùng chung prompt template + JSON schema đầu ra.

#	Server	Nhiệm vụ	API/tool bên ngoài cần
1	mcp-server-project-db	SQLite, state machine, parse script.docx, Character Bible, pre-flight IP check	Local; thư viện đọc docx
2	mcp-server-deepix	Gen ảnh hàng loạt, tải ảnh, ghi kết quả	REST API + key Deepix (nội bộ)
3	mcp-server-qc-agent	Chấm ảnh theo qc_checklist , cả 2 mode; Claude Vision + system prompt QC riêng (xem 3.2.1), provider swap được	Claude (V0: chat; V1: Anthropic API, pin version)
4	mcp-server-clipai	Gửi/poll gen video, heartbeat polling batch, parse lỗi risk-control (comment 4); chọn model theo dự án (video_model : seedance / kling / minimax)	REST API (+webhook) Clip AI (nền tảng tích hợp nhiều model)
5	mcp-server-ffmpeg	Concat, transition, mux nhạc, render	FFmpeg local
6	mcp-server-music	Music Brief, gen 3 bản nháp, thư viện/upload nhạc (xem 3.5)	API nhạc có license thương mại (chọn sau)

Ngoài ra: Claude Director (Bước 1, 3, Music Brief), Streamlit Dashboard (V1), Figma (mockup giao diện).

3.3b Thông tin về Clip AI (bổ sung 2026-09-19)

Clip AI là nền tảng tạo video và âm thanh AI tích hợp cho team (all-in-one), gộp nhiều model: **Seedance, Kling, MiniMax** (video), **Seed Audio, ElevenLabs** (âm thanh/giọng), kèm thư viện game asset. Tính năng: tạo/chỉnh video đa phương thức, đạo diễn 3D, thoại, âm thanh và nhạc, thiết kế và nhân bản giọng nói.

Hệ quả cho thiết kế: 1. **Không chỉ Kling**: bộ chạy Bước 4 có cấu hình video_model theo dự án (seedance / kling / minimax), truyền vào provider. Bộ lọc kiểm duyệt (risk control) và cú pháp prompt khác nhau theo model → blacklist IP và content_moderation_failures nên ghi kèm model; bộ đánh giá eval/ cần chạy lại khi đổi model. 2. **Âm thanh**: Bước 5a (nhạc) có thể dùng chính Clip AI (Seed Audio/ElevenLabs), giảm một nhà cung cấp. Cần xác nhận API có endpoint audio và điều khoản thương mại. 3. **Giọng/thoại**: tạo thoại, thiết kế/nhân bản giọng là tính năng có thể thêm ở giai đoạn sau (lồng tiếng nhân vật); chưa nằm trong V0/V1. 4. **Thư viện game asset**: phù hợp trailer game; xét dùng làm nguồn tham chiếu nhân vật/bối cảnh nhất quán khi API cho phép. 5. **Điểm cần hỏi khi xin API**: danh sách model và phiên bản có thể chọn qua API, tham số chọn model, endpoint audio, hạn mức/credit theo model, chính sách kiểm duyệt theo model.

3.4 Giao diện điều khiển hợp nhất (Unified Control Dashboard)

Nguyên tắc: 1. **Một tool duy nhất** — mọi hành động (upload script, gen ảnh, duyệt, gen video, nhạc, render) thực hiện từ Dashboard; user không phải chuyển qua lại giữa công cụ. 2. **Bố cục Stepper/Wizard theo đúng thứ tự Bước 1 → 5**, mỗi bước là 1 màn hình riêng, đầu trang luôn hiển thị tiến độ theo state machine. Cho phép quay lại xem/sửa bước trước bất kỳ lúc nào; không cho vào bước sau khi bước trước chưa đủ điều kiện (vd chưa Approve & Lock Character Bible thì chưa vào Bước 2). 3. **Mỗi khâu có bộ nút điều khiển tác vụ riêng** (bảng dưới), mỗi nút map vào state machine: Run → `queued` → `running`; Retry → `retryable` → `queued` (tăng `retry_count`, ghi `retry_reason`); Cancel → `cancelled`; Approve/Reject → ghi `review_log` (`reviewer_type`). 4. **Chỉ chạy lại phần thay đổi** (học từ ComfyUI): Reject 1 ảnh và sửa mô tả thì chỉ re-run đúng job đó (qua `parent_job_id`), không re-run các bước/cảnh không đổi. 5. **Tab Lịch sử:** xem lại các phiên bản ảnh/video đã reject của cùng 1 scene để so sánh. 6. Giữ giao diện tuyến tính (không dùng node-graph tự do như ComfyUI) vì yêu cầu sắp xếp đúng thứ tự bước.

Thanh điều khiển toàn cục (luôn hiển thị): chọn project/kịch bản; switch `operating_mode` (`auto` / `human_qc`); slider `qc_auto_pass_threshold`; Kanban tổng quan scenes theo state; nút khẩn cấp **Pause toàn bộ / Resume toàn bộ / Cancel tất cả job đang chạy**.

Bộ nút điều khiển theo từng bước:

Bước	Nút điều khiển
1. Kịch bản & phân tích	Upload script.docx · Chạy phân tích · Sửa Character Bible/bảng cảnh inline · Lưu · Duyệt & khóa · Hủy/Reset bước · Cảnh báo Pre-flight IP risk
2. Gen ảnh + QC	Gen ảnh hàng loạt (tắt cả/chọn scene) · mỗi ảnh: Approve · Reject + ghi chú · Gen lại · hàng loạt: Approve tất cả PASS · Gen lại tất cả FAIL · hiển thị mode đang áp dụng + QC score
3. Video Motion Prompt	Sinh prompt hàng loạt · Sửa prompt · Duyệt từng cái · Duyệt tất cả
4. Gen video	Bắt đầu gen batch · thanh tiến độ heartbeat (x/y) · Pause/Resume polling · Retry job fail · Cancel job đang chạy · cảnh báo IP risk
5a. Nhạc nền	Sinh/sửa Music Brief · Gen 3 bản nháp · Nghe thử · Chọn 1 · Gen lại · Upload nhạc · Không dùng nhạc
5b. Ghép & render	Chọn transition · Render Final · Preview · Tải xuống · Render lại với option khác · Nghiệm thu (Done)

Bản xem trước bằng Figma: mockup chi tiết (control bar, stepper, màn hình từng bước 1–5b, tab Lịch sử, trạng thái empty/loading/error, badge theo state machine) — user duyệt trước khi build. **Trạng thái mockup (2026-09-19):** - **HTML tương tác (đầy đủ):** [mockup/dashboard.html](#) — mở bằng trình duyệt; đủ 6 màn hình (Bước 1, 2, 3, 4, 5a, 5b) + tab Lịch sử, control bar toàn cục, đổi `operating_mode` /threshold thấy tác động lên Bước 2. Dữ liệu giả. - **Figma:** [file Figma](#) — có component TopBar, Stepper (6 biến thể), Màn hình Bước 1 (dựng bằng Figma MCP) và **bản import đầy đủ 7 màn** từ [mockup/dashboard_all.html](#) bằng

plugin `html.to.design` (frame "dashboard_all.html", các màn xếp dọc). Các màn 2–6 chỉ được dựng qua import HTML vì gói Figma Starter hết hạn mức MCP; layer là kết quả import tự động nên cần tách frame/đặt tên/componentize thủ công nếu muốn dùng làm design system. Việc tồn đọng: Stepper component đang chồng lên Screen 1 trên canvas (cần dời vị trí).

Lưu ý Streamlit khó khớp 100% pixel với Figma; nếu cần UI sát mockup thì cân nhắc frontend web riêng (quyết định mở).

3.5 Bước 5a — Nhạc nền

AI đề xuất, user quyết định: (1) Claude Director đọc Visual Mood + tổng thời lượng → sinh Music Brief (thể loại, tempo, cảm xúc, nhạc cụ, instrumental), user sửa hoặc tự viết prompt; (2) gen **3 bản nháp cho cả video** (1 track xuyên suốt, tránh đứt giữa cảnh), nghe thử và chọn/gen lại; (3) hoặc upload nhạc có sẵn/thư viện nội bộ/không dùng nhạc; (4) FFmpeg ghép (fade, ducking, căn độ dài) → preview → render. - **Suno:** hiện **không có API chính thức** (07/2026 mới mở intake form cho partner, chưa có self-serve; gói Pro/Premier chỉ dùng web/app); các "Suno API" bên thứ ba là wrapper không chính thức → rủi ro ToS/ổn định/bản quyền, không đưa vào pipeline. Dùng Suno thủ công: tạo trên web rồi upload vào slot. - `music_provider` swap được. **Ứng viên số 1 nay là chính Clip AI** (nền tảng tích hợp ElevenLabs và Seed Audio, có tạo nhạc/âm thanh) — nếu API Clip AI mở phần audio thì dùng chung 1 nhà cung cấp, 1 hợp đồng, 1 API key; nếu không thì dùng nhà cung cấp riêng có license thương mại rõ. **Chọn sau khi xác nhận API Clip AI** (chưa chặn V0).

3.6 Knowledge Base & Prompt Library (nền chất lượng Bước 1 và Bước 3)

Nguyên tắc: **không train/fine-tune model** — xây knowledge pack có cấu trúc + ví dụ mẫu + JSON schema đầu ra + bộ đánh giá, nạp vào system prompt Director; rẻ, sửa ngay được, đổi model được. - **Bước 1 (biên kịch/đạo diễn):** cấu trúc cảnh (slugline, beat, mục tiêu), bối cảnh, nhân vật/trang phục (= Character Bible), ánh sáng/màu sắc/mood, ngôn ngữ ống kính (cỡ cảnh, góc máy, tiêu cự), continuity giữa cảnh. Đầu ra JSON: scene, location, time, characters, wardrobe, mood, lighting, shot list, image prompt. - **Bước 3 (video motion prompt):** thư viện theo thể loại (hành động, kinh dị, drama, game trailer...), từ vựng chuyển động camera (push/pull/pan/tilt/orbit/handheld), tốc độ, hành động nhân vật trong 5s, cú pháp Kling image-to-video; mỗi thể loại có few-shot tốt/xấu + negative prompt. - **Nguồn tham khảo (chưa kiểm định — review chất lượng/license trước; chỉ lấy tri thức, không chạy code lạ tùy tiện):** [awesome-ai-video-prompts](#), [ai-shortfilm-prompts](#), [visual-skills](#), [SKRIPTON](#), [OpenStory](#), [awesome-llm-story-generation](#), [Kling Prompt Guide](#); cộng sách/giáo trình biên kịch-đạo diễn-quay phim do team nội dung chọn và kịch bản/storyboard các dự án đã làm. - **Quy trình:** thu thập + lọc → chưng cất knowledge pack → viết system prompt + schema → **eval set** 10–20 mẫu có đáp án do chuyên gia phim/nội dung chấm → lặp cải thiện → khóa version. Cần 1 "chuyên gia miền" duyệt. Ước tính +8–12 ngày công, chạy song song với build MCP ở V0; bộ mẫu dùng luôn để hiệu chỉnh QC checklist (Bước 2).

4. Rủi ro & mitigation tổng hợp

Rủi ro	Mitigation
Auto-reject/retry vô hạn tổn credit	<code>max_retry_count</code> + escalate to user
QC threshold “hộp đen”, không giải trình được	<code>qc_checklist</code> theo tiêu chí + threshold config được
Batch lớn cần user tức trực approve	Heartbeat polling, approve 1 lần/batch
Gen hàng loạt rồi fail vì trùng IP bản quyền	Pre-flight check ở Character Bible + blocklist nội bộ
Không rõ trạng thái job khi lỗi nửa chừng	State machine chuẩn + audit trail <code>review_log</code>

5. Quyết định đã chốt & còn mở

Đã chốt: - `qc_agent_provider` = Claude Vision (system prompt QC riêng) — xem 3.2.1 (2026-09-18) - Schema state machine, retry logic, `review_log.reviewer_type` — xem Mục 2, 3.1 - Điều khiển hợp nhất qua 1 Dashboard duy nhất, stepper theo thứ tự bước, nút điều khiển riêng từng khâu — xem 3.4 (2026-09-19) - Triển khai 2 giai đoạn: **V0** (thử nghiệm qua Claude Desktop + MCP, phương án B) rồi **V1** (Dashboard hợp nhất + Anthropic API) — xem Mục 7 (2026-09-19) - Kiến trúc core dùng chung + MCP vỏ mỏng + LLM runner hoán đổi được — xem 3.3 - Không train/fine-tune model; dùng Knowledge Base + eval set — xem 3.6

Đã chốt cho V0 (2026-09-19): - `operating_mode` mặc định V0 = `human_qc` (mọi ảnh chờ người duyệt; an toàn credit khi chưa có dữ liệu QC). Chuyển `auto` sau khi dry-run cho thấy % đồng thuận QC Agent-người đủ cao. - Ngân sách credit thử nghiệm V0: **nhỏ** (~2 kịch bản, 10–20 cảnh) — đủ dry-run tối thiểu; nếu cần hiệu chỉnh threshold sâu hơn thì xin thêm. - **Deepix:** đã kiểm tra, **có API lấy được. Clip AI/Kling: chờ kiểm tra** (chưa xác nhận có REST/Webhook) — nếu chỉ có Web thì dùng Playwright (+1–2 tuần cho `mcp-server-clipai`). Việc cần làm: kiểm tra Clip AI, đồng thời nắm response schema lỗi risk-control.

Còn mở, KHÔNG chặn V0 (chốt trước V1): | Quyết định | Ghi chú | |—|—| | Nguồn truy cập Claude cho V1 | Gói Claude (seat Enterprise/Pro/Max) và Anthropic API (Console, API key, tính tiền theo token) là 2 sản phẩm/billing tách biệt; seat không tự sinh API key. Dashboard tự gọi Claude bằng code nên V1 cần API key từ Console org công ty (không nên dùng đăng nhập seat cho ứng dụng tự động — cần xác nhận với admin/điều khoản Anthropic). Việc cần làm: hỏi admin Enterprise (a) có Console org không, (b) hạn mức token, (c) chính sách data cho ảnh nhân vật. | | `music_provider` | Suno không có API chính thức. Ưu tiên dùng audio của Clip AI (ElevenLabs/Seed Audio tích hợp) nếu API cho phép; xác nhận license thương mại. Xem 3.5 | | Framework Dashboard | Streamlit (mặc định) hay web frontend riêng nếu cần UI sát mockup Figma |

Còn mở — cần chốt trước khi code V0 (chặn Tuần 1-2): | Quyết định | Vì sao cần | Ai chốt | |—|—|—| | `operating_mode` mặc định (`auto` hay `human_qc`) cho từng loại nội dung/dự án | Quyết định luồng review có block hay không — ảnh hưởng thiết kế Dashboard & MCP QC Agent | Bạn + team sản xuất nội dung | | Deepix và Clip AI/Kling có REST API + Webhook thật không, hay chỉ có giao diện Web | Nếu chỉ có Web, Bước 2 & 4 phải dùng Playwright automation thay vì gọi API — kiến trúc MCP server khác hẳn (chậm hơn, dễ vỡ hơn khi UI đổi) | Team dev nội bộ giữ API Deepix/Clip AI | | `qc_auto_pass_threshold` khởi điểm (0.85 là giả định, chưa có dữ liệu thật) | Threshold sai (quá lỏng/chặt) làm hỏng hiệu quả cả 2 mode ngay từ đầu | Chốt tạm ở dry-run, hiệu chỉnh bằng dữ liệu thật Tuần 3 | | Ngân sách credit cho giai đoạn thử nghiệm

(Deepix + Clip AI/Kling) | Dry-run + retry loop đều tốn credit thật; cần biết giới hạn trước khi chạy batch | Bạn / người giữ ngân sách |

6. Checklist chuẩn bị — cần có gì trước khi bắt đầu Tuần 1

Truy cập & tài liệu: - [] API endpoint + API Key/Authentication của **Deepix** (REST) - [] API endpoint + API Key/Authentication của **Clip AI / Kling AI** (REST + Webhook nếu có) - [] Xác nhận rate limit / quota của cả 2 API (ảnh hưởng trực tiếp `batch_poll_interval_seconds`) - [] Tài liệu response schema của Kling AI khi bị content-moderation chặn (để `mcp-server-clipai` parse đúng lỗi “Failure to pass the risk control system” — Comment 4)

Hạ tầng vận hành: - [] Máy chạy pipeline (PC vận hành) cài **Claude Desktop App** + Python runtime - [] File `claude_desktop_config.json` trỏ tới các MCP servers local - [] Môi trường Python cho lớp core + 6 MCP servers (`project-db`, `deepix`, `qc-agent`, `clipai`, `ffmpeg`, `music`) — venv/dependencies - [] (V1) API key Anthropic Console cho Director/QC gọi bằng code — xem Mục 5 - [] FFmpeg cài local, kiểm tra version + codec cần dùng (transition, re-encode) - [] Ổ đĩa đủ dung lượng cho `/images/`, `/videos/` khi chạy batch lớn

Dữ liệu & nội dung: - [] 2 kịch bản mẫu (5-10 cảnh/kịch bản) để dry-run Tuần 3 - [] Danh sách IP/nhân vật dự kiến dùng trong dự án thật — để build blocklist ban đầu trước khi chạy batch thật (Comment 4) - [] Bộ tiêu chí `qc_checklist` cụ thể (không chỉ “đúng nhân vật” chung chung — cần liệt kê rõ: góc mặt, trang phục, tỉ lệ khung hình, độ phân giải tối thiểu...) để Claude Vision QC Agent chấm điểm nhất quán

Con người / vai trò: - [] Người build 5 MCP servers (dev) - [] Người thiết kế & build Streamlit Dashboard - [] Người review thẩm mỹ thật (đóng vai “User” ở 3 điểm kiểm duyệt) trong dry-run — cần người này để đo % Claude Vision QC Agent đúng/sai thực tế - [] “Chuyên gia miền” biên kịch/đạo diễn để duyệt Knowledge Base và chấm eval set (Mục 3.6) - [] Nguồn tri thức nền: sách/giáo trình biên kịch-đạo diễn-quay phim, kịch bản + storyboard các dự án cũ (Mục 3.6) - [] Người giữ quan hệ với team dev nội bộ cấp API Deepix/Clip AI

7. Lộ trình triển khai: V0 → V1

Giai đoạn V0 — Thử nghiệm (Claude Desktop + MCP, ≈ 3–4 tuần)

Mục tiêu: chứng minh pipeline end-to-end chạy được, đo chất lượng QC, hiệu chỉnh threshold, chốt Knowledge Base/prompt. Các bước cần Claude (Director Bước 1/3, QC Bước 2, Music Brief) chạy qua chat Claude Desktop với prompt template chuẩn; các thao tác còn lại gọi qua MCP tool. Dashboard V0 tối giản (xem trạng thái + duyệt ảnh) hoặc bỏ qua.

Tuần 1 — Hạ tầng & chốt quyết định mở - Lấy API docs + key Deepix và Clip AI/Kling; xác nhận có REST/Webhook hay chỉ Web (nếu chỉ Web → Playwright, +1–2 tuần). - Cài Claude Desktop + Python; test kết nối MCP cơ bản (“hello world”). - Chốt `operating_mode` mặc định, ngân sách credit thử nghiệm. - Khởi động Knowledge Base (Mục 3.6): thu thập + lọc nguồn, chọn chuyên gia miền. - **Deliverable:** API access hoạt động, môi trường sẵn sàng, các quyết định chặn V0 đã chốt.

Tuần 2 — Core + MCP servers - Implement schema SQLite + state machine **trước tiên** trong lớp core dùng chung. - Build MCP `project-db`, `deepix`, `qc-agent` (cả 2 nhánh `auto` / `human_qc`), `clipai` (heartbeat polling, parse lỗi moderation), `ffmpeg`; `music` có thể để sau (upload nhạc thủ công ở V0). - Song song: viết system prompt + JSON schema cho Director/QC, tạo eval set. - **Deliverable:** chạy được end-to-end 1 cảnh qua 5 bước bằng tool call.

Tuần 3–4 — Dry-run & hiệu chỉnh - Dry-run 2 kịch bản mẫu (5–10 cảnh) ở cả 2 mode; người review thật chấm song song với QC Agent → tính % đồng thuận → hiệu chỉnh `qc_auto_pass_threshold` (0.85 chỉ là giả định). - Lắp cải thiện prompt theo eval set; build blacklist IP v1 (case Wonder Woman + danh sách nhân vật dự kiến). - **Cổng chuyển V1:** ≥1 video hoàn chỉnh end-to-end, threshold đã hiệu chỉnh bằng dữ liệu, prompt/checklist khóa version.

Giai đoạn V1 — Hợp nhất trên 1 Dashboard (≈ +3–4 tuần, cần API key Anthropic)

- **Song song đầu V1:** Figma mockup chi tiết (Mục 3.4) → user duyệt trước khi build.
- Build Dashboard theo Mục 3.4 (stepper 5 bước, control bar toàn cục, nút điều khiển từng khâu, Kanban, tab Lịch sử, cảnh báo IP, tiến độ heartbeat) gọi lớp core.
- Thay LLM runner từ chat thủ công sang Anthropic API (cùng prompt template + schema).
- Build `mcp-server-music` + Bước 5a (Music Brief, 3 bản nháp) sau khi chọn `music_provider`.
- Đóng gói 1-Click Startup, runbook (xử lý job `failed` bị escalate, chỉnh threshold an toàn), Prompt Templates bàn giao.
- **Deliverable:** hệ thống bàn giao vận hành được từ 1 giao diện duy nhất, có tài liệu.

Ước tính nỗ lực (giả định đã có đủ API key/docs/môi trường/kịch bản mẫu; dev Python có Claude Code hỗ trợ)

Hạng mục	Ngày công
Schema + state machine + project-db	3–4
deepix	2–3
qc-agent (+ checklist prompt)	3–4
clipai (heartbeat, moderation, retry)	4–5
ffmpeg	2–3
music	2–3
Knowledge Base + eval set (Mục 3.6)	8–12
Dashboard Streamlit (màn hình 5 bước + control bar + lịch sử)	8–10
Figma mockup (song song)	2–3
Tích hợp E2E + dry-run + hiệu chỉnh threshold	4–5
Đóng gói 1-click + runbook + prompt templates	3
Tổng	~41–55 ngày công

→ **1 dev: ≈ 8–11 tuần; 2 dev (1 backend/MCP + Knowledge Base, 1 dashboard): ≈ 5–7 tuần.** Rủi ro kéo dài: chờ API Deepix/Clip AI; Deepix/Clip AI chỉ có Web (Playwright, +1–2 tuần); hiệu chỉnh QC threshold cần nhiều vòng; chất lượng Knowledge Base phụ thuộc chuyên gia miền.

8. Định nghĩa “Hoàn thành” (Definition of Done) cho toàn dự án

- ☐ Chạy được ít nhất 1 video hoàn chỉnh end-to-end từ script.docx đến FINAL_VIDEO.mp4 qua Dashboard (V1), không cần chỉnh code tay
- ☐ Cả 2 `operating_mode` đều test được và cho kết quả đúng như thiết kế ở Mục 3.2
- ☐ `max_retry_count` + escalate hoạt động đúng khi cố tình cho 1 job fail liên tục
- ☐ Pre-flight IP check cảnh báo đúng với ít nhất 1 case đã biết (Wonder Woman) trước khi chạy batch thật
- ☐ Batch > 100 video chạy được qua heartbeat polling mà không cần user approve từng cái
- ☐ Runbook + Prompt Templates đã bàn giao cho bộ phận sản xuất nội dung

9. Trạng thái triển khai (cập nhật 2026-09-19)

Đã build (69 unit test pass, không cần API ngoài): - `core/` — schema SQLite, state machine job, luồng QC `auto / human_qc`, `retry/escalate`, `Pause/Resume/Cancel-all`, script parser (docx → cảnh/nhân vật), validators + lưu JSON của Director/QC, Character Bible + khóa, Pre-flight IP check + blacklist seed, motion prompt (Bước 3), FFmpeg command builders, prompt bundles. - `mcp_servers/` — `project-db`, `qc-agent`, `ffmpeg-studio`, `clipai` (vỏ mỏng bọc `core`; `clipai` mới chạy bản giả lập); hướng dẫn `docs/V0_SETUP.md`. - `dashboard/app.py` — Dashboard Streamlit skeleton: stepper 7 tab theo thứ tự bước, control bar toàn cục, nút điều khiển từng khâu, lịch sử; các bước cần Claude tạm dán JSON tay. - `prompts/`, `knowledge/` (v0.2: nền tảng đạo diễn, hướng dẫn 7 thể loại, lỗi ảnh AI, từ vựng motion; người duyệt tạm là chủ dự án), `data/qc_checklist.json`, `samples/` (2 kịch bản mẫu cho dry-run). - `eval/` + `core/evalset.py` — bộ đánh giá Director: 12 mẫu (3 golden làm few-shot, 9 held-out), bộ chấm tự động phần khách quan, phiếu duyệt thẩm mỹ; hướng dẫn ở `eval/README.md`.

Đang chờ điều kiện bên ngoài: - Adapter thật cho Deepix (Bước 2) và Clip AI/Kling (Bước 4) — tạm gác theo yêu cầu. Bộ chạy Bước 2 và Bước 4 (`core/runner.py`: `submit`, `heartbeat polling`, tải kết quả, log risk control, `retry`, `concurrency`, `pause/cancel`) đã xong và kiểm thử với provider giả lập; chỉ còn viết adapter theo interface `VideoProvider / ImageProvider` khi có API. - Cài FFmpeg trên máy để render thật. - API key Anthropic (V1) để thay “dán JSON tay” bằng LLM runner tự động. - `music_provider`; chạy vòng đánh giá thật (người duyệt tạm = chủ dự án, sau này chuyên gia miền) và dry-run V0 để hiệu chỉnh `prompt/threshold`.

Tài liệu nguồn: `Quy_Trinh_Auto_Pipeline_Full1.docx` (kèm 4 comment góp ý, đã phân tích ở Mục 2).